

Last updated: March 7, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 7 [\[LINK\]](#)

Tasks part A: Understanding special files

1. Try the following commands as a non-privileged user. What does each do? How do the files f1-f3 compare? How do they compare to /dev/urandom? Remember you can get output from /dev/urandom using cat or dd. If you use cat, make sure to pipe it to less!

- o `cp /dev/urandom f1` (hit Ctrl+C right away to avoid a huge file)
- o `sudo cp -a /dev/urandom f2`
- o `sudo mknod f3 c 1 9`

Note: If you see any command running for more than a few seconds, hit Ctrl+C right away, check the produced file (often it's very large) and delete it if not needed.

“-a” preserves all properties of the device node, so you are creating an identical copy (equivalent to calling `mknod`, hence the need for root). By contrast, without “-a” `cp` just tries to read from the device node (invoking the driver's `read()` function) and write to another created regular file. As `/dev/urandom` never sends an end-of-file (EOF) you will get infinite data, hence filling up your disk (that's why you must stop it right away).

2. Make named pipes using `mknod` and `mkfifo`. Use them to simulate `ls | wc` using just the '>' and '<' operators.

```
wc < mypipe & ls > mypipe
```

Here assume you have a named pipe in your current directory called `mypipe`. The “&” operator runs the first one in the background and starts the second so that both can execute at the same time (not blocked). You can also use two terminals as was done in one lecture.

This shows while internally “|” is just implemented using an unnamed pipe, you can also manually achieve the equivalent using the named pipe, which is exposed as a file (in this case `mypipe`) in the file system.

3. Use `mknod` to make a copy of your current terminal's tty - `/dev/tty` (Hint: consider how you did something similar with `/dev/urandom` above). You can name it `mytty`. Examine its characteristics using `stty --file=mytty`. Do the same for the original tty.

The main purpose here is to show you that when both the major number and the minor number are the same, you are technically operating the same device instance. Reflected from your tty, you should be able to see (from task 4 below) that disabling echo for `mytty` is equivalent to disabling echo for your original tty.

Note: the exact behavior of multiple device nodes with identical major/minor varies with device drivers. In previous tutorials, you might have seen that your current tty is also `/dev/pts/1` (which `/dev/tty` points to). But if you do the same to it, the PTMX/PTS driver won't allow it (Input/output error).

```
ls -l /dev/tty
```

```
crw-rw-rw- 1 root tty 5, 0 Mar  8 00:51 /dev/tty
```

```
sudo cp -a /dev/tty mytty or sudo mknod mytty c 5 0, to make a copy of your current tty.
```

Now, you can `stty -F mytty -echo` (this is equivalent to `stty -echo`)
To get echo back, you need to blindly type `stty echo`, and hit Enter.

4. Run `stty --help` to see what you can do with `stty`. Try disabling local echo. How does the shell behave with echo disabled? How can you restore echo without logging out and back in? Recall we did this in the class.
See above.

Tasks part B: A simple kernel module

Download the source for this [simple module](#), unpack, and build it by typing "make". You can safely ignore the warning of "Skipping BTF generation...".

1. Install the module using "`sudo insmod simple.ko`". The hello message is recorded in the kernel logs. How do you view the kernel logs? How many ways are there to view them?
There are multiple ways. You can use `dmesg` to check kernel messages since the most recent boot, or view `/var/log/syslog` or `/var/log/kern.log`. If you're interested, you can try to find out how such log files are gradually archived (something, something.1, then compressed: something.2.gz, etc.).
2. Check to see that the module has been loaded. How do you do this?
`lsmod`
3. Remove the module from the kernel. What did you do?
`sudo rmmod simple`

Tasks part C: A character device kernel module (driver)

Download the source for [ones](#), a kernel module implementing a character device that outputs an unbounded string of "1"s. Build, compile, and run it as before.

1. What kernel messages does the module generate? Does it create any new files (other than `/dev/ones`)? If so, where? (Hint: search in `/sys`)
`sudo find /sys -name ones`
2. What happens when you "cat" the device `/dev/ones`? How can you limit the output in more than one way?
You will be flooding your screen.
`cat /dev/ones | less`
`cat /dev/ones | more`
`cat /dev/ones | head -100`
3. Add more `printk()`'s (where you see needed) to find out at which point which functions in `ones.c` are called and for which purpose.

Tasks part D: Getting process information from a module

Download the source [newgetpid.c](#). Build and run it as before. Hints for the questions below:

<https://elixir.bootlin.com/linux/latest/source/include/linux/sched.h>, <https://elixir.bootlin.com/linux/latest/source/arch/x86/include/asm/current.h>, <https://elixir.bootlin.com/linux/latest/source/include/linux/cred.h>

1. What type is "current"? How can you figure this out?

```
struct task_struct *
```

This is about your code reading skill. We will read it briefly in the class.

2. Modify newgetpid.c so that it creates a device file /dev/describe rather than /dev/newgetpid.

```
#define DEVICE_NAME "describe"
```

3. Make /dev/describe output the calling process's parent ID (ppid), user ID (uid), group ID (gid), effective user ID (euid), and effective group ID (egid).

Not considering namespaces:

```
snprintf(message, 150, "Your PID is %d; your PPID is %d; your UID is %d; your GID is %d; EUID is %d; your EGID is %d!\n", thepid, task_ppid_nr(current), current->cred->uid.val, current->cred->gid.val, current->cred->euid.val, current->cred->egid.val);
```

Considering namespaces:

```
snprintf(message, 150, "Your PID is %d; your PPID is %d; your UID is %d; your GID is %d; EUID is %d; your EGID is %d!\n", thepid, task_ppid_nr(current), from_kuid_munged(current_user_ns(), current->cred->uid), from_kgid_munged(current_user_ns(), current->cred->gid), from_kuid_munged(current_user_ns(), current->cred->euid), from_kgid_munged(current_user_ns(), current->cred->egid));
```

Using bpftrace to monitor kernel module events

Note: you are not required to write about your experience for this part in the submitted work, but you should still do it, which can help you understand eBPF better (see how simple but still powerful an eBPF program can be).

Download this tiny code [kmsnoop.bt](https://github.com/tytso/kmsnoop.bt) and make it executable with `chmod 755 kmsnoop.bt`.

Now, you can open a separate SSH session and run it with `sudo ./kmsnoop.bt`. It will wait for events. When you do any "insmod" or "rmmod" in the other session, you will see it outputs a line of message.

If you want to avoid opening another SSH window, you can use tmux (a terminal multiplexer) by just typing tmux and hit enter.

To create a new session: press Ctrl-B and C

To move between sessions: press Ctrl-B and N (or P)

How could this save you from doing any printk's in your kernel module code? You can also `sudo bpftrace -l | grep` for your needed kernel functions (e.g., `printk`) to hook to.

There are generally three types of "events" your bpftrace code can hook to: `tracepoints`, `kprobes` and `uprobes`. Since we are looking at kernel events, we can use either `tracepoints` or `kprobes`. They are different in the sense that `kprobes` are about "raw" access to kernel functions with original context, and hence may or may not exist (due to kernel changes). `Tracepoints` are implemented/supported by kernel developers as explicit and static attach points, hence more reliable. `Tracepoints` have well-defined data structures for you to extract. You can see details with: `sudo bpftrace -lv "[any tracepoint name]"`

We will also briefly show this in the class.
